Jeux C#

*Vaisseau **ZINZIN***

Sommaire

- Univers du jeu
- Déroulement d’une partie
- UML et spécificités
- Gestion de Projet

Univers du jeu

Un **Zinzin** est dans une situation critique, après un terrible incident il s'est retrouvé dans un vaisseau spatial inactif. Pour rentrer chez lui, il doit **retrouver la clé** de démarrage parmi le dédale d'étages et de modules du vaisseau ! Cependant, celui-ci risque de s'auto-détruire dans les **10 minutes**, il faut faire vite !

Exigences Fonctionnelles

- 10 étages, de types lieu de vie, salle des machines et poste de pilotage.
- Des modules dans les étages : moteur, coffre, couchage, pilotage et téléporteur (pouvant échouer !).
- Enregistrement du score, “meilleur joueur” et Mode Test (pour faire un test rapide des fonctionnalités du jeu) et chronomètre.

Exigences Techniques

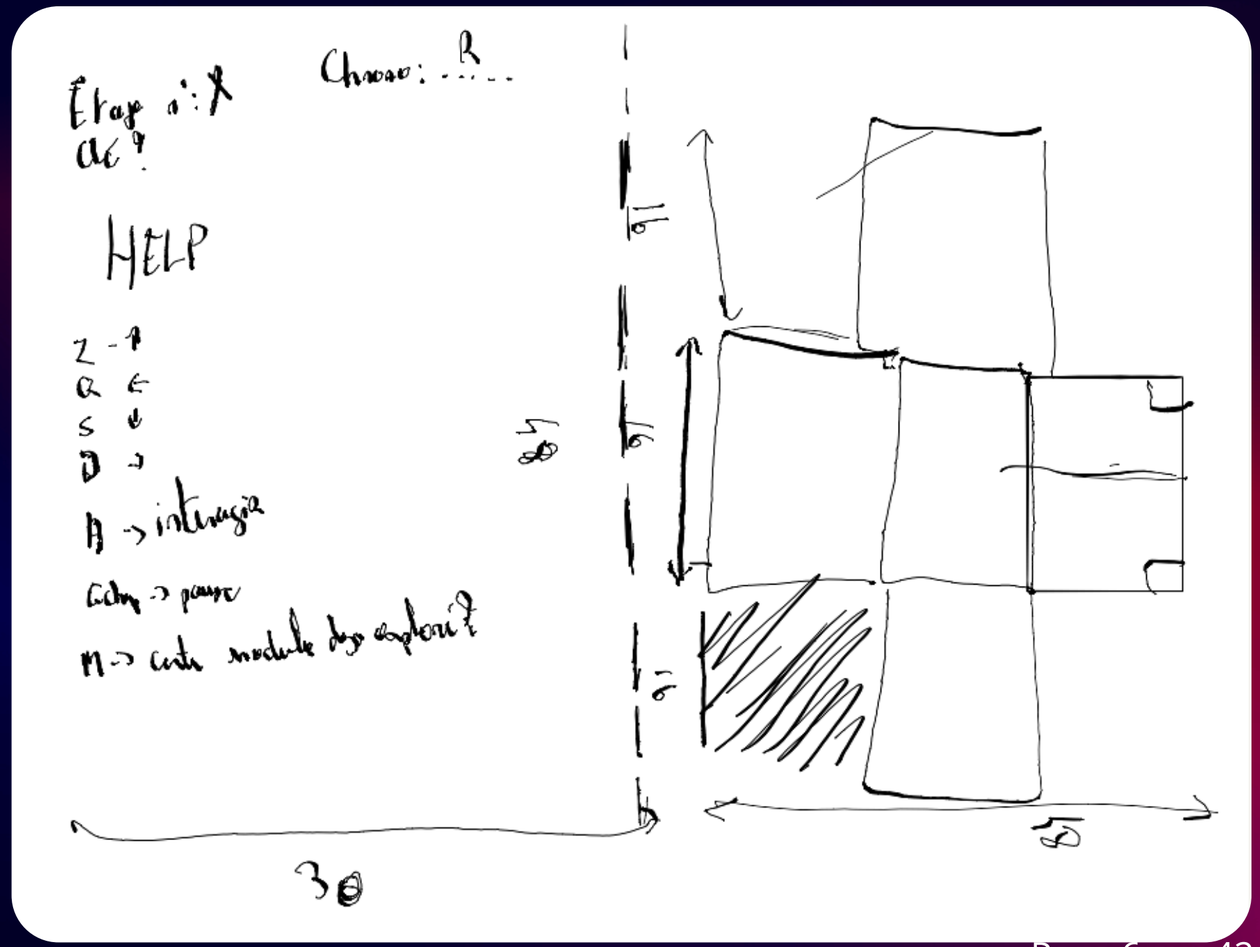
- Application Console C# avec .NET 9
- Utilisation des mécanisme de la POO : associations, héritages, classes abstraites (notament pour les étages et les modules)
- Indentation correcte, respect des conventions et rendu sans erreurs (ni warnings)

Maquettage

Une des première étape a été d'essayer de chercher comment on allait représenter le jeu sur la console.

Nous voyons comme **visuel** un jeu **divisé** en **deux** avec à droite le jeu et à gauche des aides et indications par rapport au jeu.

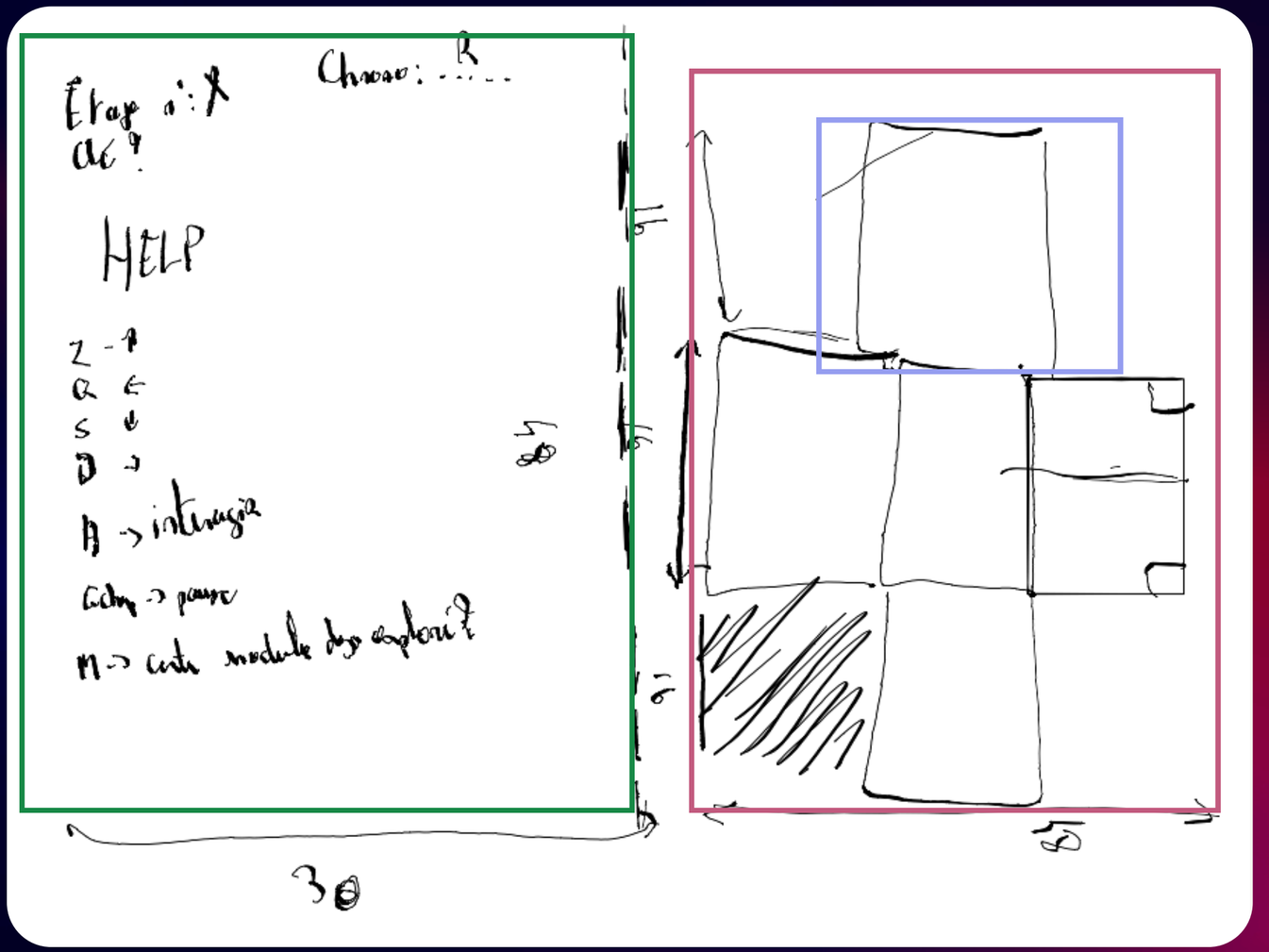
Nous avons tous de suite imaginé en lisant le sujet qu'un module etait une boite dans laquelle nous pouvions se déplacer.
C'est d'ailleurs vers ce concept du module que nous sommes partis. Nous expliquerons plus en détails dans les prochaines slides.



Maquettage

- Vue du dessus, étages par étages
- Deplacement libre entre les modules
- Division entre le jeu et l'UI

UI Module Etage

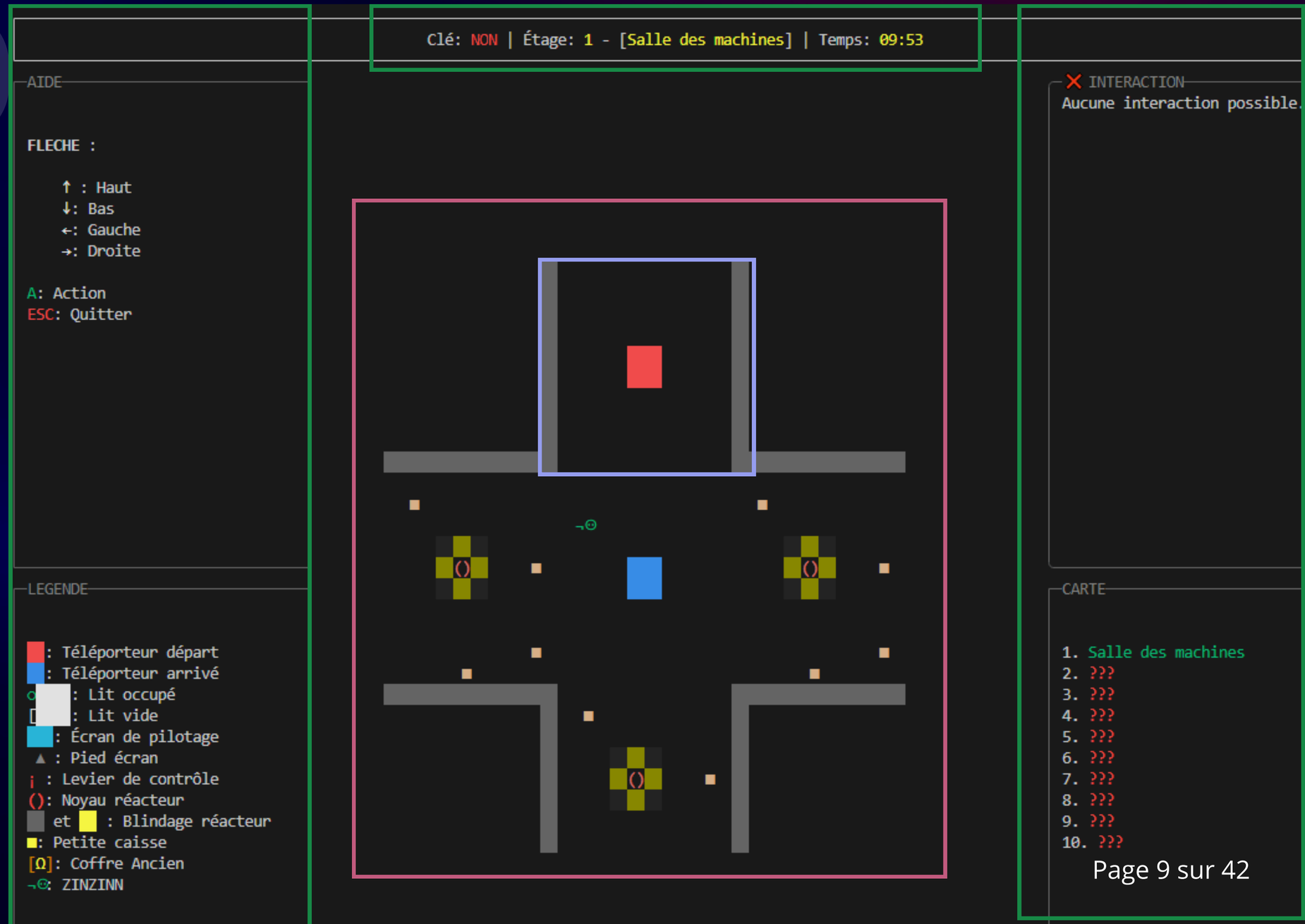


Inspiration

Ce rendu etait une de nos inspirations et notre ambition pour le visuel de notre jeu console.

Rendu Final

- UI
- Module
- Etage



Deroulement d'une partie

1. **Ecran titre**
   Choix du mode de jeu
2. **Choix du nom et de l'étage d'apparition**
3. **Exploration du vaisseau**
4. **Interaction avec les élements des modules pour trouver la clé**
5. **Victoire en rapportant la clé au bon endroit et enregistrement du score**

Boucle principale

```
while (chrono.enCours && temps < 10 min && !victoire)
lire touche -> deplacement / interaction -> verifier victoire -> rafraichir UI
```

1 Ecran titre

Elements Principaux

- Choix entre le mode normal (10 étages) et le mode test (3 étages)
- Contexte et objectifs

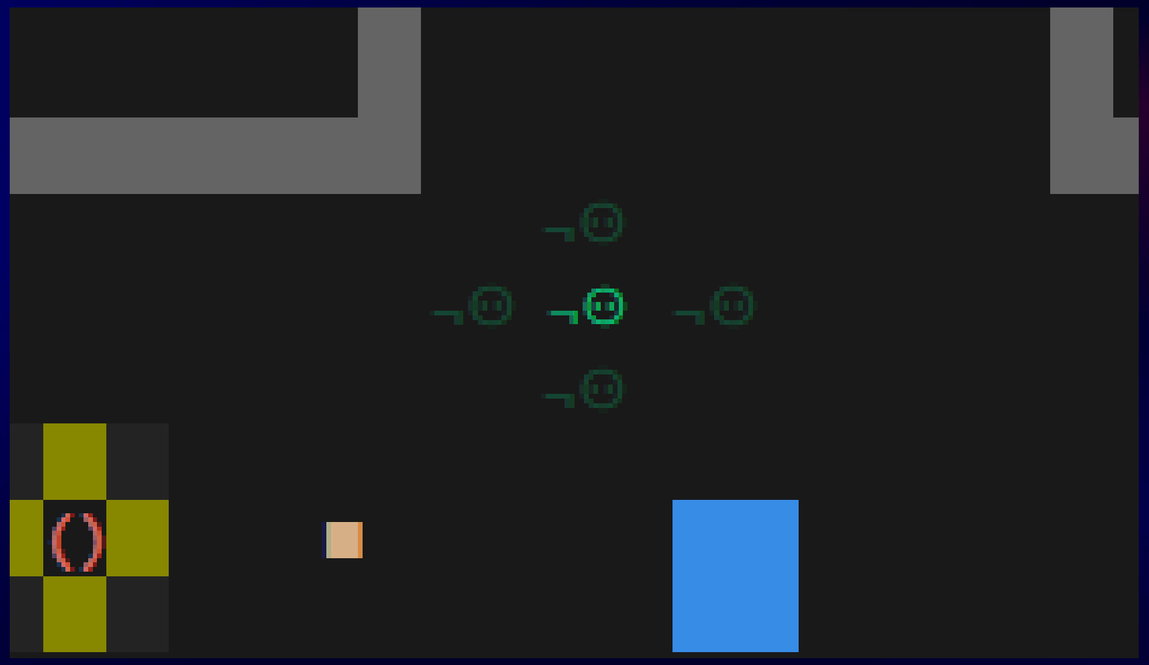
2 Choix

```
RENSEIGNEMENT POUR LA MISSION

Quel est votre nom?

Joueur

Quel étage voulez-vous être ?


  1. Poste de pilotage
> 2. Salle des machines
  3. Lieu de Vie
  4. Lieu de Vie
  5. Poste de pilotage
  6. Poste de pilotage
  7. Salle des machines
  8. Poste de pilotage
  9. Lieu de Vie
  10. Poste de pilotage
  11. Annuler
```

Elements Principaux

- Choix du nom (affiché sur le classement)
- Choix de l'étage d'apparition (si la téléportation fonctionne)

3 Exploration

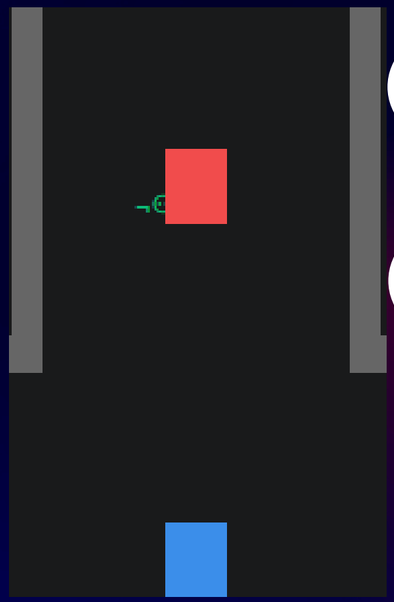


Deplacement au sein d'un étage

- Via les flèches directionnelles
- Gestion des colisions
- Deplacement libre entre les modules

3 Exploration

```
TELEPORTEUR DEPART
Choisir destination :

▶1.
2. Lieu de Vie
3.
4.
5.
6.
7.
8.
9.
10.
Esc : Annuler
```



Téléporteur de départ

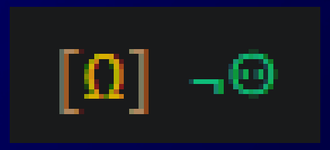
Téléporteur d'arrivée

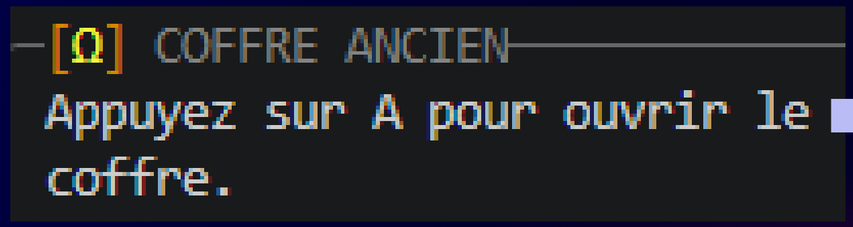
```
CARTE

1. ???
2. Lieu de Vie
3. ???
4. ???
5. ???
6. ???
7. ???
8. ???
9. ???
10. ???
```

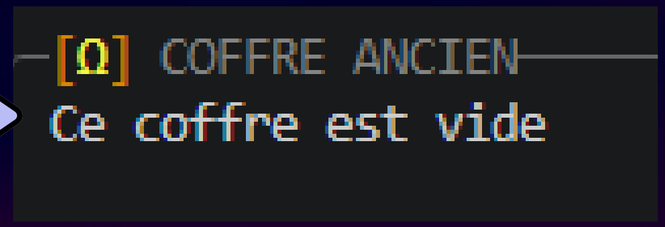
Déplacement entre les étages

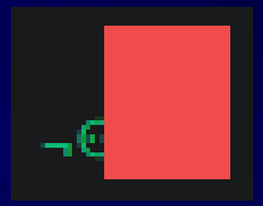
- Via les téléporteurs de départ (apparition à celui d'arrivée de l'étage d'arrivée)
- Carte du vaisseau s'actualisant au fur et à mesure de la découverte

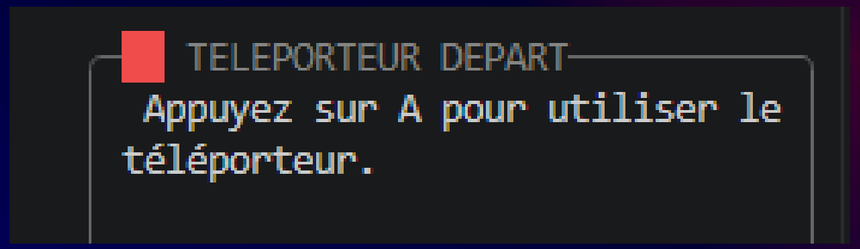
4 Interactions

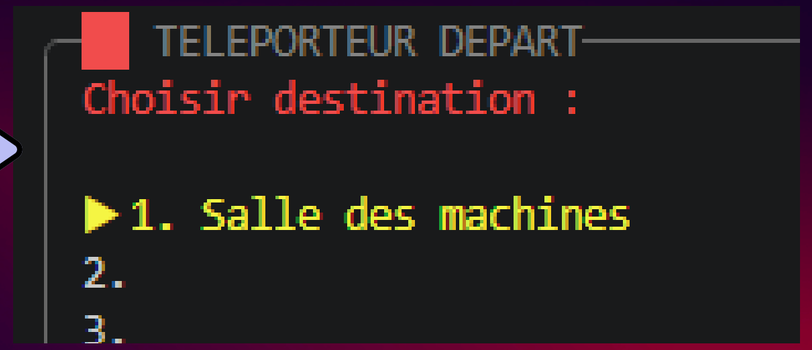








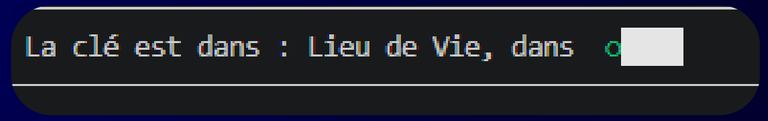


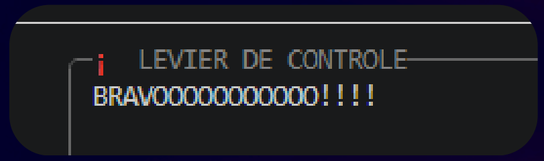


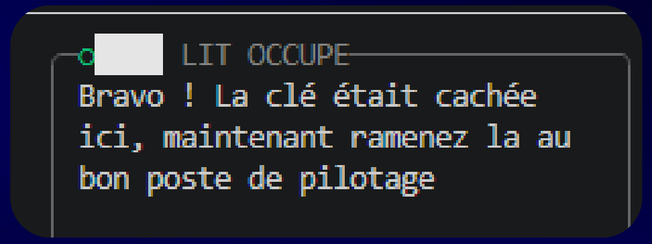
Interaction avec les éléménts

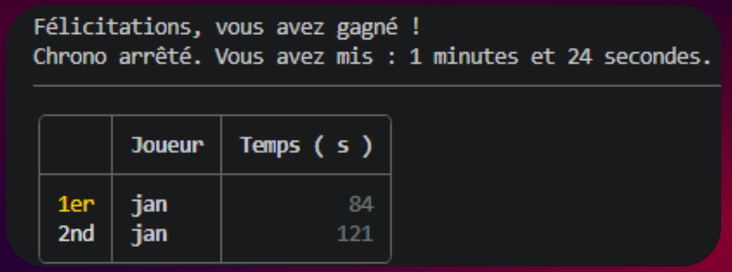
- Via le bouton “A” en étant en contact avec l’élement
- Chaque élément à une interaction dédiée

5 Victoire









Conditions de Victoire

- En moins de 10 minutes, le joueurs doit avoir intéragi avec l'élement où est caché la clé et le bon levier de pilotage.
- Dans le mode test, la cachette est indiquée dans l'en-tête
- Les scores sont ensuite affichés (disponibles également dans l'écran titre)

5 Score

Meilleurs temps du mode Test

	Joueur	Temps (s)
1er	jan	84
2nd	jan	121

Meilleurs temps du mode Normal

	Joueur	Temps (s)
1er	jan	156
2nd	Zoe	220

```
gxk;156
Wlb;220
```

Enregistrement des scores

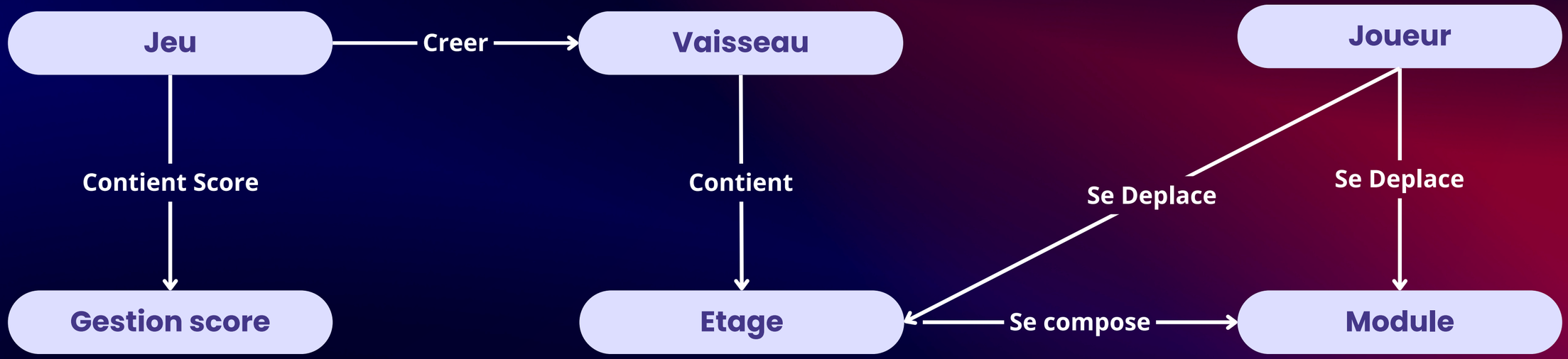
- Les scores des mode test et normal sont séparés. Ils consistent simplement au temps mis à relancer le vaisseau, en seconde.
- Les scores sont stockés dans des fichiers CSV de manières cryptés (via césar) pour eviter des potentielles triches.

Construction du jeu

La suite de cette présentation présente nos choix techniques pour le développement, ainsi qu'un focus plus précis sur les elements centraux du jeu.

Diagramme de classes UML

Le diagramme ci-dessous résume la structure objet du projet. Il s'agit d'une version très simplifiée du réel diagramme, qui sert à montrer les relations entre les principales différentes classes



UML réel

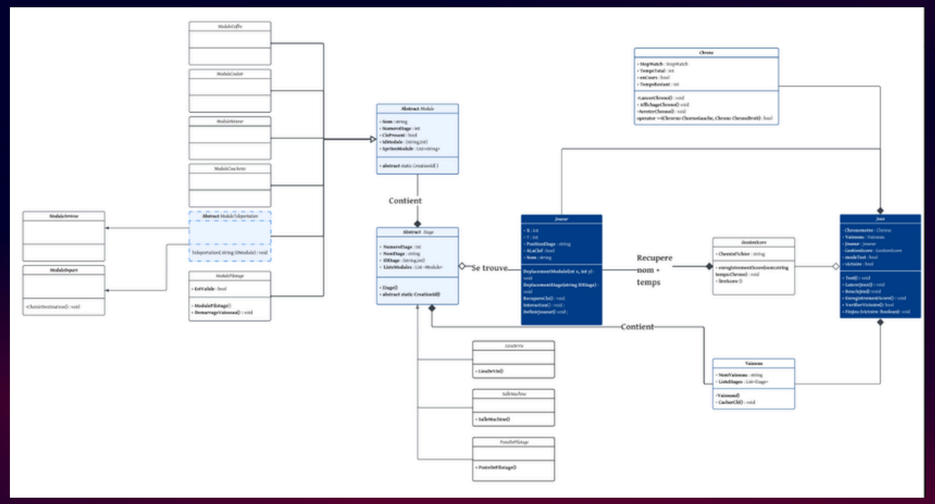
Vous retrouverez l'UML complet dans le lien ci-dessous, étant bien trop conséquent pour tenir dans cette présentation.

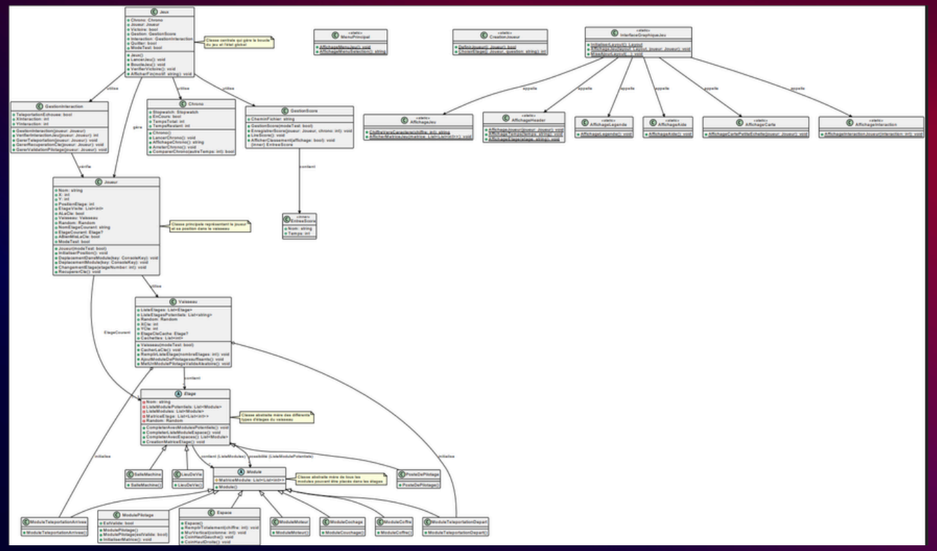
Lien vers le diagramme UML final

UML prévisionnel

Ce que nous pouvons montrer en revanche, c’est la différence entre le premier UML, fais avant de commencer tout code et celui fait une fois le code terminé. Nous sous-estimions grandement la quantité de classes et de liaisons nécessaires à un tel projet.

[Lien vers le premier diagramme UML]





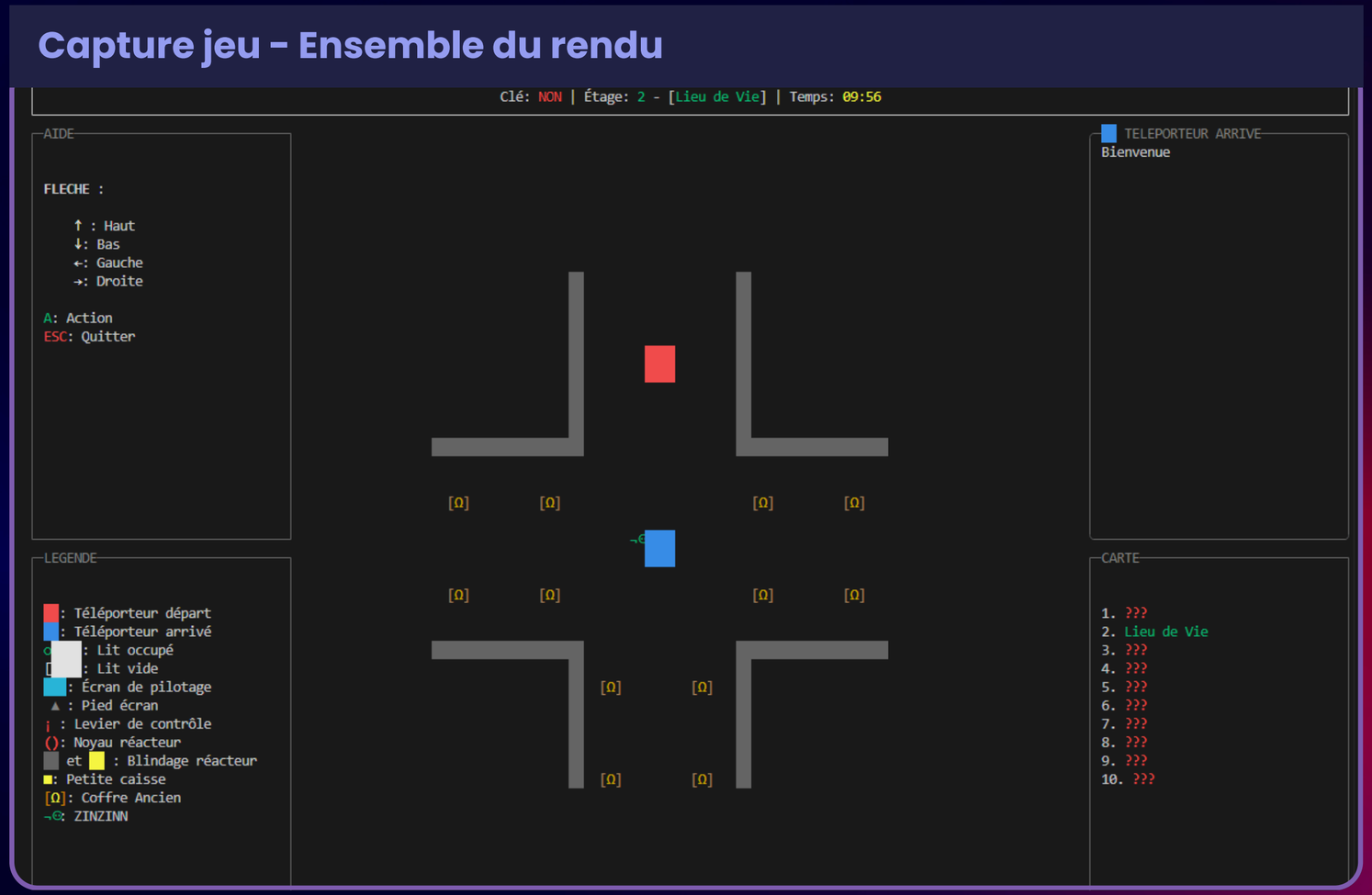
Focus sur les élements

Nous sommes rapidement passés sur les différents éléments du jeu lors du **déroulement de la partie**, nous allons à present vous les présenter en détails

Interface

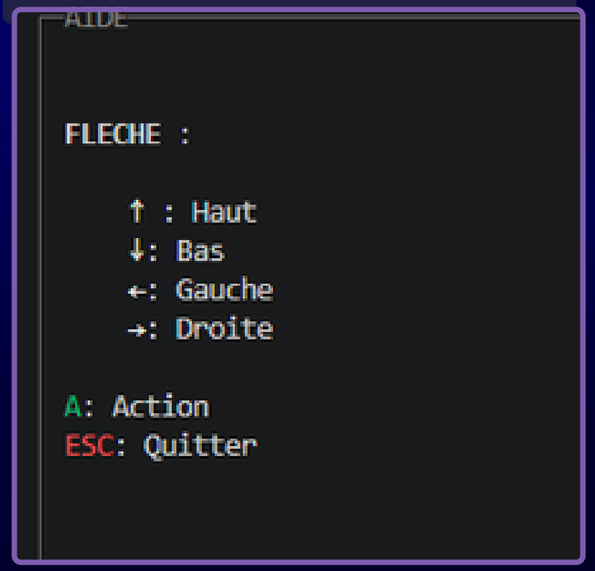
Le jeu en lui même est une application console auxquelles on a rajouté l’extension Spectre.Console pour obtenir une interface avec des panneaux. Cela a permis une inteface plus claire et esthétique, améliorant l’experience de jeu de l’utilisateur.

Capture jeu - Ensemble du rendu

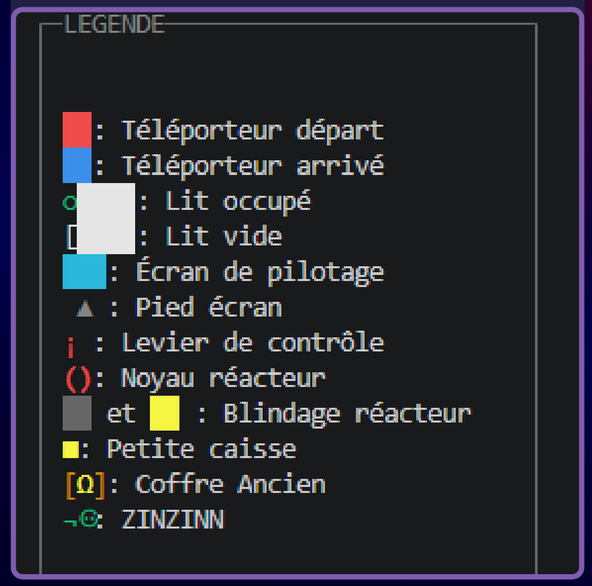


Interface

Aide



Légende



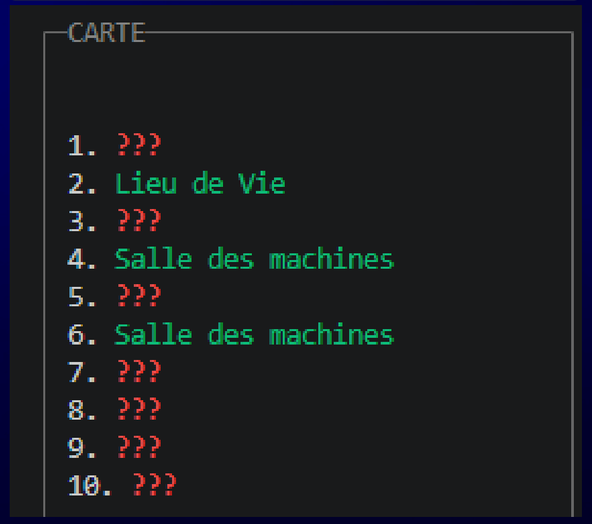
La partie **aide et légende** permet aux joueurs de savoir n'importe quand :

- Les interactions claviers qu'il peut avoir pour intéragir dans le jeu
- La définition et les élements constitutifs du jeu

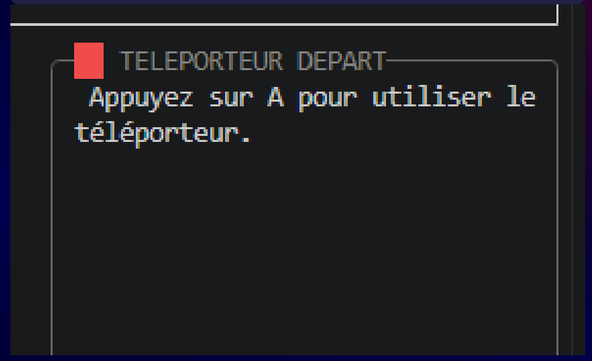
Grâce à cet **affichage,** qui est présent tout le long du jeu, le joueur ne peux pas se perdre lors de la partie.

Interface

Carte



Intéraction



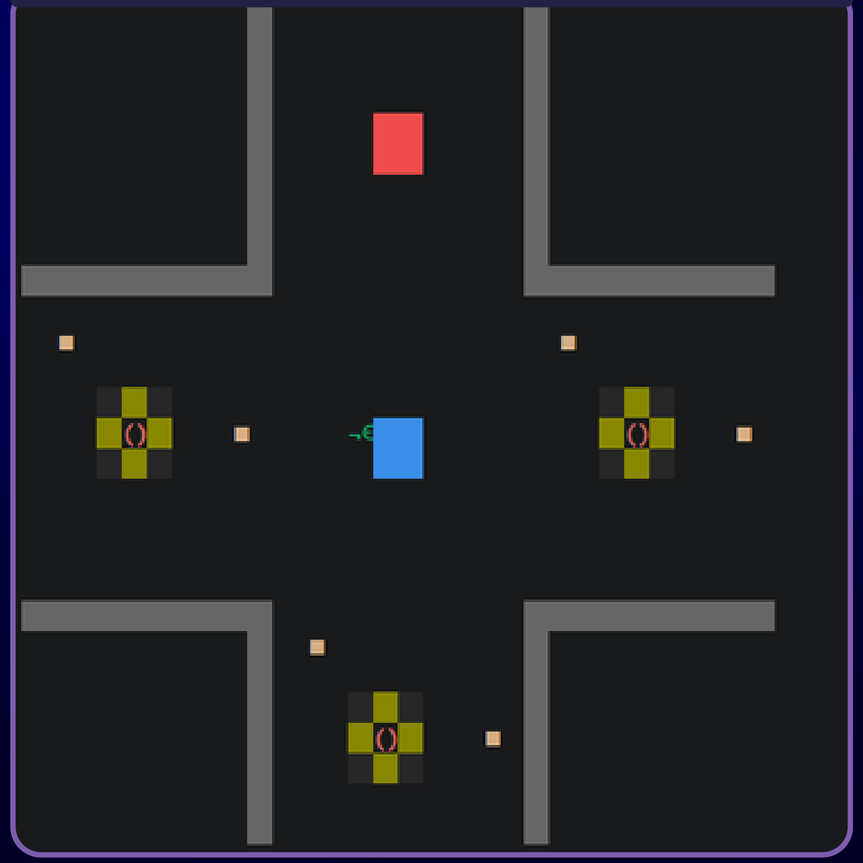
La partie **carte et interaction** permet aux joueurs de savoir n'importe quand :

- Les **interactions possibles** lorsqu'il est à coté d'un objet
- Le **nom des étages** visités avec le nom du type de l'étage.

Grâce à cet **affichage le** joueur peut essayer de déduire s'il lui reste des étages à visiter et avec quels élements il peut agir.

Interface

GamePlay



Barre d'état du joueur

```
Clé: NON | Étage: 2 - [Salle des machines] | Temps: 09:59
```

La partie **Barre d'état et Gameplay** permet aux joueurs de voir n'importe quand :

- Si il possède la clé ou non
- Dans quel étage il est
- Combien de temps il lui reste
- Le visuel du jeu avec son placement par rapport à l'étage

Vaisseau

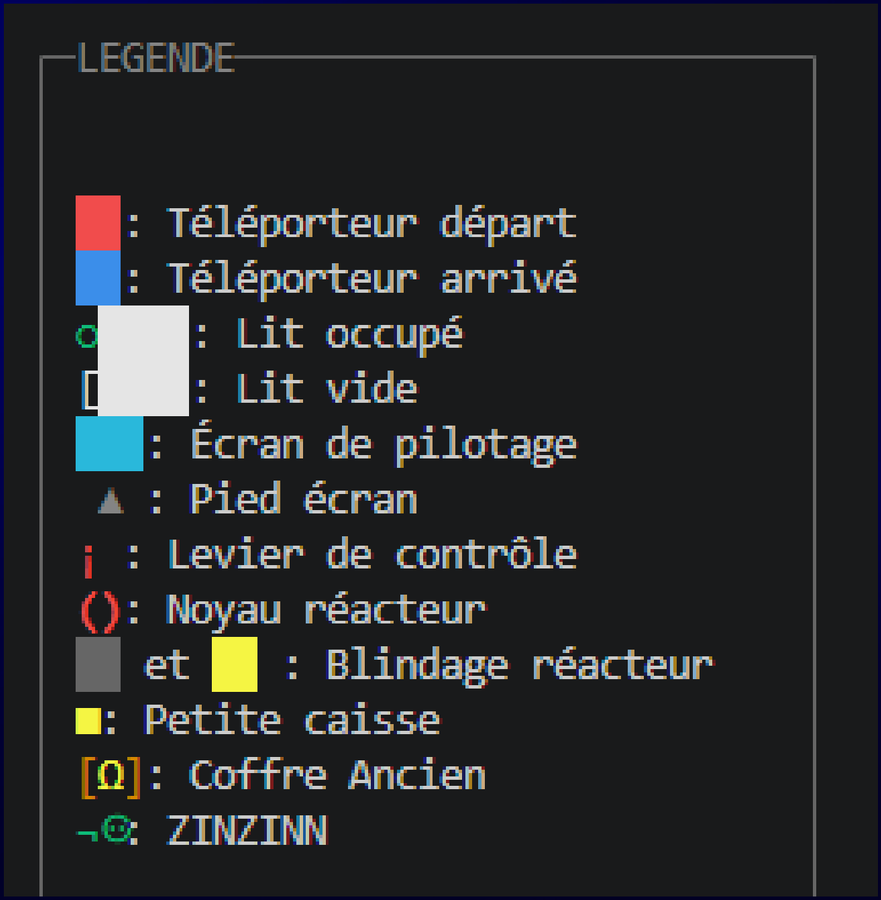
Chaque vaisseau est **unique** et composé de :

- 10 etages en mode normal, 3 en mode test. A chaque lancement du jeu, un vaisseau **aléatoire** est construit, avec toujours au moins un étage de chaque type (lieu de vie, salle des machines, poste de pilotage) et au moins 2 modules de pilotage

- Chaque étage est une grille 30x30 composée de 9 blocs de 10x 10, appelés **modules**. 5 sont remplis et le reste, c'est du vide spatial.

- A l'interieur de chaque module, il y a des objets avec lesquels on peut intéragir.

Elements constitutifs des modules



Voici une liste des élèments disponible dans le jeu avec lesquels on peut intéragir durant la partie.

Trouver la clé :

- Lit occupé
- Lit vide
- Caisse
- Coffre

Inserer la clé :

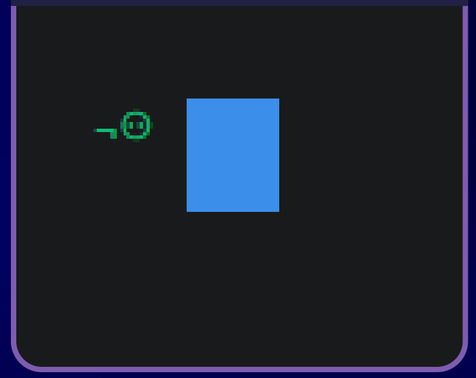
- Levier de contrôle

Téléportation :

- Depart de la téléportation
- Arrivé de la téléportation

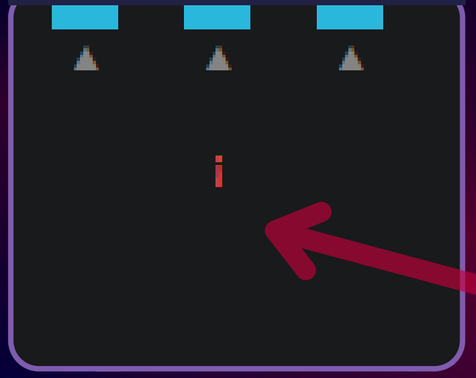
Modules

Teleportation: Arrivé

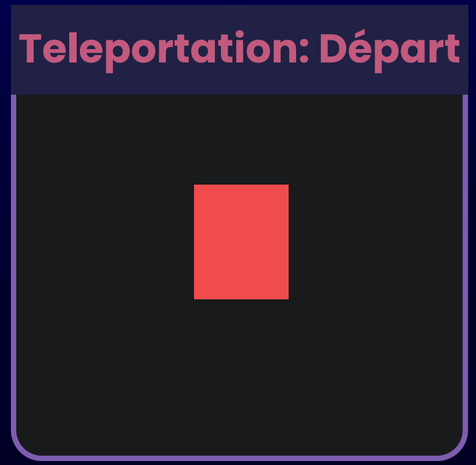


Moteur

Pilotage

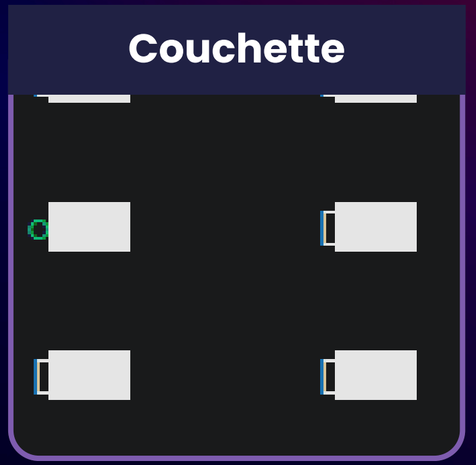


Teleportation: Départ



Coffre

Couchette

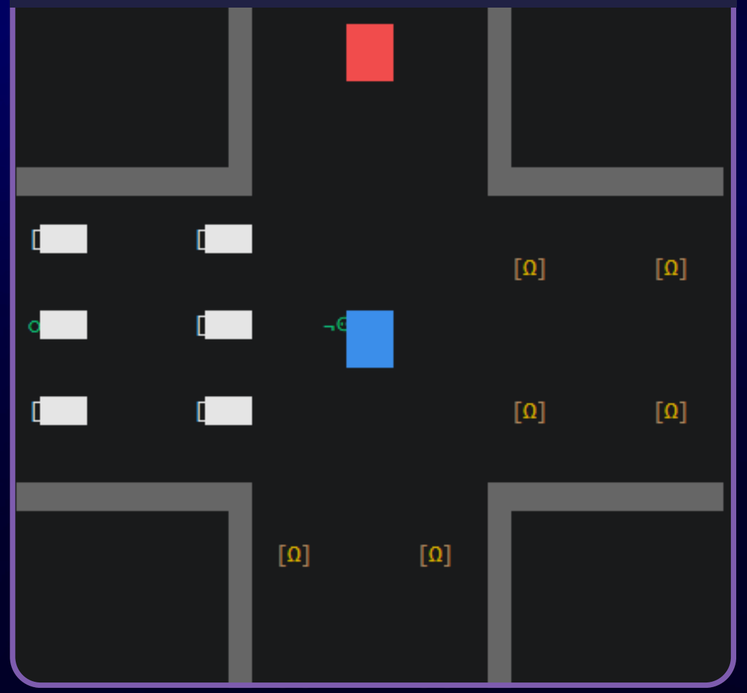


- Voici les 6 modules possibles.
- Nous pouvons trouver la clé dans les modules :
  - Moteur
  - Coffre
  - Couchette
- La clé doit etre inséré dans le levier dans le module pilotage

Etages

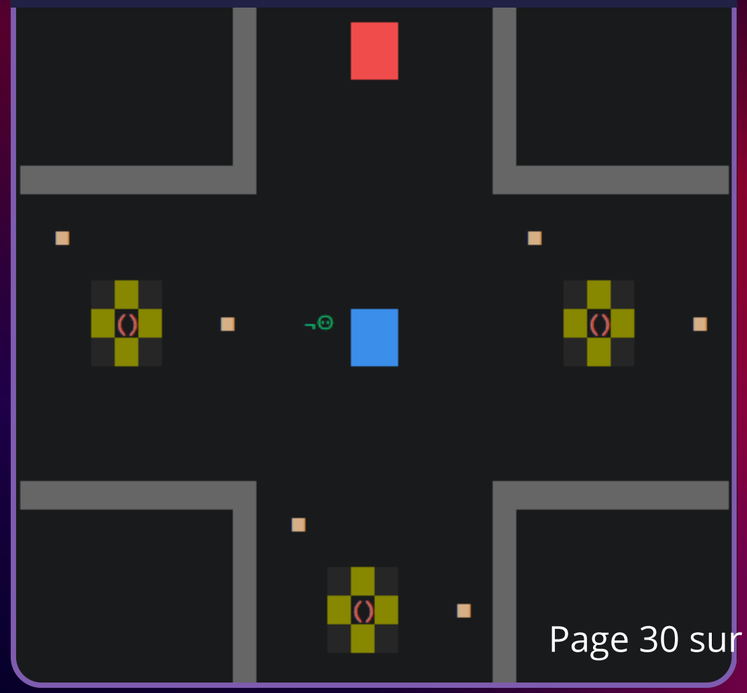
Chaque étage est généré aléatoirement selon une liste de module possible dépendant de sa nature. Les seuls modules constant sont les téléporteurs d'arrivée et de départ

Lieu de vie -
Module possible : Couchette/Coffre



Poste de pilotage -
Module possible : Pilotage/Machine

Salle machine -
Module possible : Machine/Coffre



Classement et Scores

Affichage Classement

	Joueur	Temps
1st	Zoe	67
2nd	Zoe	67
3rd	Zoe	67
4	Bob	95
5	Alice	120
6	Carol	150
7	gvj	600
8	e	600

A la fin de la partie gagnée, le nom du joueur est enregistrer et son temps aussi.
Le nom est crypté en césar.
Ils sont ajouté dans un fichier CSV.
Le fichier est ensuite lu et affiché trié par temps croissant et affiche les 10 meilleurs

EXPLICATION
Spectre.Console

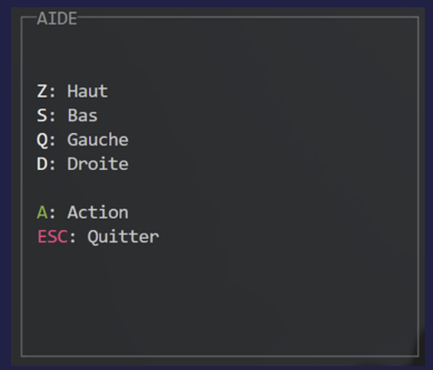
Pour le rendu du jeu, nous avons voulu utilisé une librairie disponible sur internet permettant d'avoir un visuel plus complet avec des **couleurs** et surtout avec la possibilité de **découper l'écran** en plusieurs parts permettant **d'actualiser** uniquement ce dont on avait besoin durant la partie : Jeux/Interaction/Barre de renseignement.

Spectre.Console

Zoom sur les fontionnalités

Panels

Utilisé pour encadrer des zones : aide, légende, carte, interaction et barre d'état



Table

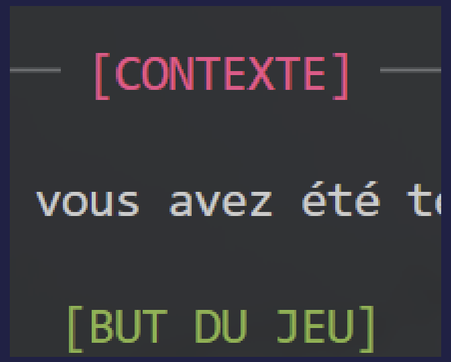
Utilisé pour afficher les meilleurs joueurs. Permet de creer des tableaux.

Spectre.Console

Zoom sur les fontionnalités

Markup

Permet de colorer le texte avec cette syntaxe : [green] [/], [yellow] [/], [red] [/] etc.



Progress

Outil visuel qui donne l'impression d'une progression avant le jeu. Permet juste une transition

Spectre.Console

Zoom sur les fontionnalités

SelectionPrompt

Permet de créer des selecteurs pour des choix : menu et destinations

```
> 1. Lancer le jeu
  2. Mode test
  3. Quitter
```

FigletText

Utilisé au début pour le lancement de l'affichage. Permet d'afficher le titre ZINZIN en plus grand c est comme une police d'écriture.

Spectre.Console

Structure de l'écran

La partie la plus importante dans le code pour l'affichage est la partie Layout + Live.

Le **layout** découpe l'écran en zones nommées.

Le Live permet ensuite de mettre à jour uniquement les zones nécessaire pendant la boucle du jeu.

Découpage du Layout pour le jeu

Spectre.Console

Structure de l’écran

A droite, nous pouvons voir l’initialisation des layouts au début de la partie.
Durant la partie, seulement les layout :

- Header
- EcranJeu
- Carte
- Interaction

seront mis à jour c’est les seuls qui changent durant la partie.

Initialisation début de la boucle jeu

```
var layout = InterfaceGraphique.InitialiserLayout();
layout["Aide"].Update(InterfaceGraphique.Aide());
layout["Legende"].Update(InterfaceGraphique.Legende());
```

Initialisation dans la boucle de jeu

```
while (chrono.enCours && chrono.stopwatch.Elapsed.TotalSeconds < chrono.TempsTotal && !victoire)
{
    layout["Header"].Update(InterfaceGraphique.Header(joueur, chrono));
    layout["EcranJeu"].Update(InterfaceGraphique.Jeu(this));
    layout["Carte"].Update(InterfaceGraphique.Carte(joueur));
```

Organisation de l'équipe

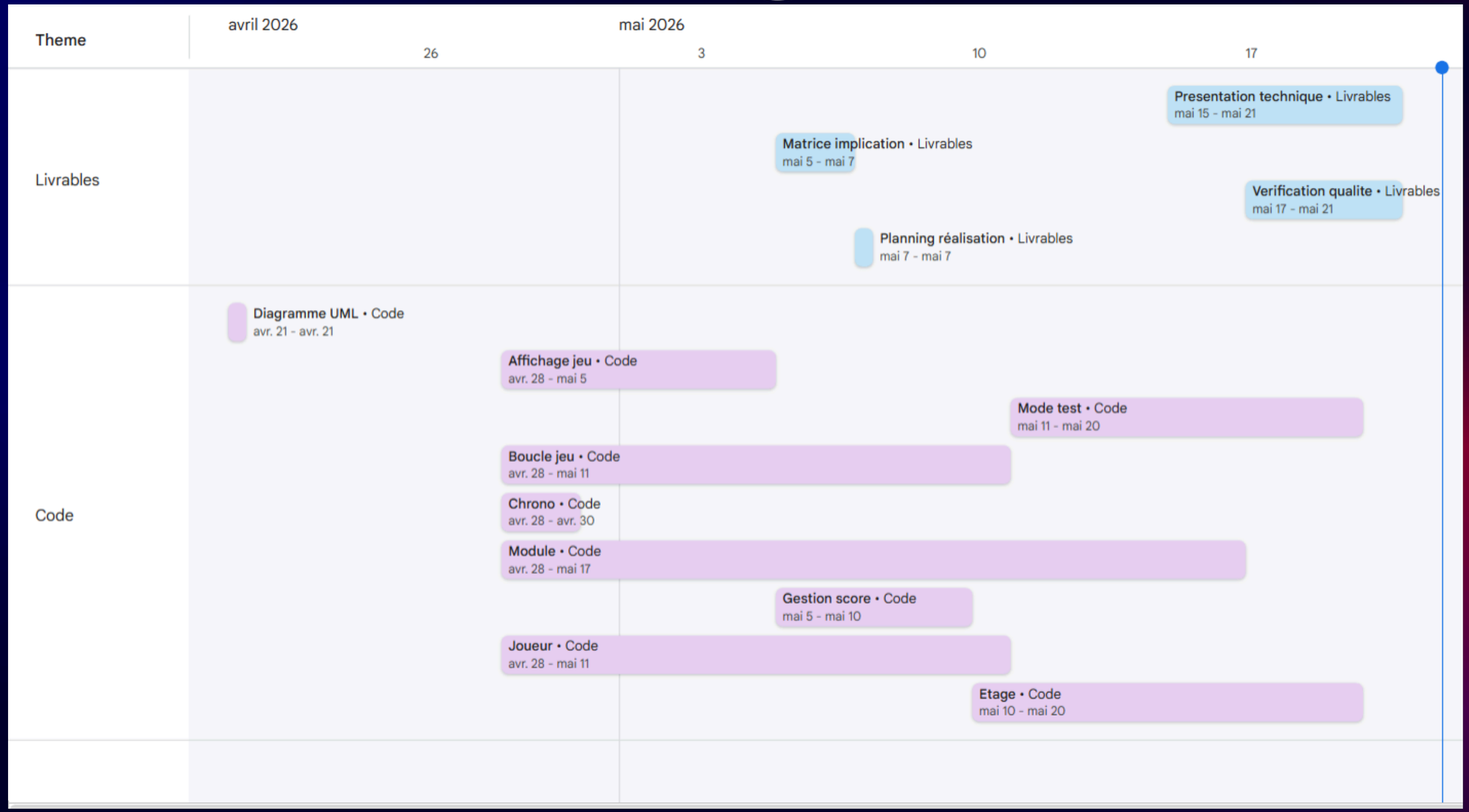
Le projet a été réalisé en binome avec github comme support de partage de code. Le travail a été divisé en plusieurs étapes : Construction UML/ Modélisation / Affichage / Gameplay / Tests/ Document de rendu

Matrice d'implication

Nous avons essayé de découper au mieux le projet pour avoir une répartition la plus équitable possible.

Nom du projet		Prograv	Date	22/05/2026
Numéro	**Membre 1**	**Membre 2**		
Noms	*Jan De Graeve*	*Zoé Schliefer Berlioux*		
Repartition :	50,00%	50,00%		
Tâches	Pourcentage du projet	Responsable	Avancement	
CONCEPTION JEUX				
Mode test	5,00%	Jan Zoe	Fait	
Diagramme UML	15,00%	Jan Zoe	Fait	
Boucle Jeu	10,00%	Zoe	Fait	
Affichage Jeu	12,50%	Jan	Fait	
Chrono	5,00%	Zoe	Fait	
Module	5,00%	Jan	Fait	
Etage	5,00%	Jan	Fait	
Vaisseau	5,00%	Jan	Fait	
Gestion Score	5,00%	Zoe	Fait	
Joueur	10,00%	Jan Zoe	Fait	
LIVRABLES				
PowerPoint	15,00%	Jan Zoe	Fait	
Matrice implication	2,50%	Zoe	Fait	
Planning de réalisation	2,50%	Zoe	Fait	
Verification Qualité code : *indentation commentaire, convention ...*	2,50%	Zoe	Fait	
	100.00%			

Planning


Theme
avril 2026
mai 2026
26
3
10
17
Livrables
Presentation technique • Livrables
mai 15 - mai 21
Matrice implication • Livrables
mai 5 - mai 7
Verification qualite • Livrables
mai 17 - mai 21
Planning réalisation • Livrables
mai 7 - mai 7
Code
Diagramme UML • Code
avr. 21 - avr. 21
Affichage jeu • Code
avr. 28 - mai 5
Mode test • Code
mai 11 - mai 20
Boucle jeu • Code
avr. 28 - mai 11
Chrono • Code
avr. 28 - avr. 30
Module • Code
avr. 28 - mai 17
Gestion score • Code
mai 5 - mai 10
Joueur • Code
avr. 28 - mai 11
Etage • Code
mai 10 - mai 20

Bilan

Pour faire le bilan du projet on a utilisé la méthode des 4L qui permet d'avoir différentes perspectives.

LIKED
Ce qu'on a le plus aimé

Appliquer concretement la POO

Produire un beau visuel

Passer de l"idée au concret

LEARNED
Ce qu'on a appris

Découvrir : spectre console

S'appropier des élements de code (switch, static class)

LACKED
Ce qui nous a manqué

Du temps

Des connaissances sur :

- L'optimisation du code
- Le game design

LONGED FOR
Ce qu'on désire améliorer

Essayer de développer le jeu sur Unity

Faire des perturbations dans le jeu

Bonne partie !

Lien Readme pour le lancement : READMe